

FORMATO ESTÁNDAR DE PROYECTOS DE DESARROLLO DE SOFTWARE

Metodología Spec-Driven Development (SDD)

MUELLE DIGITAL

Nombre del proyecto: Muelle Digital

Tipo de proyecto: Aplicación web — Marketplace de experiencias ecoturísticas fluviales

Institución / Empresa: _____ (completar)

Integrantes: Esteban Merlano (Líder, Backend y Despliegue) · Keyner Trujillo (Frontend) · Andres Rangel (Base de Datos) · Jhon Escobar (QA)

Director / Tutor: _____ (completar)

Fecha: Junio de 2026

Versión del documento: 1.0

1. Introducción

Contexto del problema

El ecoturismo fluvial en Barrancabermeja (avistamiento de manatíes, pesca artesanal, paseos en canoa y aviturismo) se ofrece de manera informal, sin canales digitales que conecten a los turistas con los guías y pescadores locales.

Planteamiento del problema

No existe una plataforma centralizada para descubrir, reservar y pagar estas experiencias; los turistas no saben a quién contactar, no hay precios claros ni mecanismos de confianza, y los pagos son informales.

Justificación

Una plataforma digital formaliza la actividad, da visibilidad y reputación a los guías locales, brinda seguridad y facilidad de reserva a los turistas, e impulsa la economía de la región.

Objetivos generales y específicos

Objetivo general: desarrollar una plataforma web tipo marketplace para reservar y pagar experiencias ecoturísticas fluviales.

- Permitir reservas digitales con control de cupos.
- Integrar pagos en línea (Stripe) y métodos locales simulados.
- Crear un sistema de reputación (reseñas en tiempo real).
- Geolocalizar el punto de encuentro en un mapa.
- Formalizar a los guías mediante verificación del administrador.

Alcance del proyecto

MVP fullstack desplegado en producción: catálogo con filtros, reservas, pagos (Stripe en modo prueba), reseñas, mapas, verificación de guías y panel de administración. Quedan fuera del alcance la app móvil nativa y el cobro real en producción.

2. Marco Teórico

Ingeniería de software

Disciplina que aplica procesos sistemáticos —análisis, diseño, implementación, pruebas y despliegue— para construir software de calidad, mantenible y confiable.

Desarrollo guiado por especificaciones (SDD)

Enfoque en el que las especificaciones (requisitos, historias de usuario y contratos de API) se definen y aprueban antes de programar, y el código se valida contra ellas, asegurando trazabilidad.

Tecnologías relacionadas

Aplicaciones web de página única (SPA), APIs REST, bases de datos relacionales, autenticación basada en tokens (JWT), pasarelas de pago y mapas basados en datos abiertos (OpenStreetMap).

Arquitecturas de software

Modelo cliente-servidor desacoplado; arquitectura por capas en el backend (rutas, controladores, acceso a datos); despliegue serverless en la nube.

3. Metodología Spec-Driven Development (SDD)

Descripción de la metodología

Se definieron primero las especificaciones (historias de usuario, modelo de datos y contrato de la API) y a partir de ellas se implementó y validó el sistema.

Principios fundamentales

- La especificación precede y guía al código.
- Trazabilidad entre cada requisito y su módulo.
- Control de versiones de documentación y código.
- Validación continua contra las especificaciones.

Fases del proceso

Definición → Arquitectura/Especificación → Desarrollo por sprints → Pruebas (QA) → Despliegue y entrega.

Control de cambios y trazabilidad

Git con flujo GitFlow (ramas por rol), gestión de tareas en ClickUp y un documento maestro versionado. Cada historia de usuario está mapeada a sus endpoints y pantallas.

4. Análisis de Requisitos

Requisitos funcionales

Resumen de los principales requisitos funcionales (ver tabla):

ID	Requisito	Prioridad	Estado
RF01	Registro de usuarios	Alta	Implementado
RF02	Inicio de sesión (JWT)	Alta	Implementado
RF03	Búsqueda y filtros de experiencias	Media	Implementado
RF04	Reserva con control de cupos	Alta	Implementado
RF05	Pago en línea (Stripe)	Alta	Implementado
RF06	Reseñas y calificaciones	Media	Implementado
RF07	Verificación de guías (admin)	Alta	Implementado
RF08	Panel de administración	Media	Implementado

			o
--	--	--	---

Requisitos no funcionales

- Seguridad: contraseñas con bcrypt, JWT, conexión SSL y control de acceso por rol.
- Usabilidad: interfaz responsive con modo claro/oscuro.
- Rendimiento: índices en la base de datos y despliegue serverless.
- Disponibilidad: alojamiento en la nube (Vercel + Neon).
- Mantenibilidad: código modular y documentado.

Historias de usuario

Se redactaron 13 historias de usuario para los actores turista, guía y administrador. Ejemplo: "Como turista quiero reservar un recorrido para asegurar mi cupo".

Casos de uso

Principales: Registrarse/Iniciar sesión, Buscar y reservar experiencia, Pagar reserva, Publicar experiencia (guía), Aprobar guía (admin) y Calificar experiencia.

Criterios de aceptación

Ejemplos: contraseña de mínimo 6 caracteres; la reserva valida cupos disponibles; la reseña usa puntuación de 1 a 5 con una sola reseña por usuario y experiencia; el guía solo publica tras ser aprobado.

5. Especificación del Sistema

Arquitectura general

Frontend en React que consume una API REST en Node/Express, la cual persiste los datos en PostgreSQL (NeonDB). La autenticación viaja como token JWT en la cabecera de cada petición.

Diagramas UML

Diagrama de casos de uso (actores: turista, guía, administrador) y diagrama de secuencia del flujo de reserva y pago. (Ver Anexos.)

Modelo entidad-relación

Seis entidades: usuarios, experiencias, reservas, ratings, pagos y solicitudes_guia, con relaciones 1:N e integridad referencial; modelo normalizado a 3FN. (Diagrama en Anexos.)

Especificación de APIs

La API está documentada con OpenAPI 3 y Swagger UI, disponible en /api/docs. Agrupa endpoints de autenticación, experiencias, reservas, ratings, pagos, solicitudes y administración.

Prototipos de interfaz

Pantallas implementadas: landing, login/registro, catálogo (lista-detalle), detalle con mapa y reseñas, paneles de guía y administrador, e historial de reservas.

6. Diseño Técnico

Tecnologías utilizadas

- Frontend: React 18, Vite, TailwindCSS, React Router, Axios, MapLibre GL.
- Backend: Node.js, Express, JWT, bcrypt, Stripe, driver pg.
- Base de datos: PostgreSQL en NeonDB.
- Despliegue: Vercel (frontend + backend serverless). Documentación: Swagger.

Estructura del proyecto

backend/ (config, middleware, controllers, routes, database) y frontend/ (api, context, components, pages). Cada capa con una responsabilidad única.

Patrones de diseño

Arquitectura por capas (rutas/controladores), middleware de autenticación, Context API para el estado global, componentes reutilizables y acceso a datos centralizado con SQL parametrizado.

Seguridad y rendimiento

Contraseñas con bcrypt, JWT por rol, claves en variables de entorno (no en el código), consultas parametrizadas y SSL. Rendimiento con índices en la BD y despliegue serverless escalable.

7. Plan de Desarrollo

Cronograma

Ocho semanas: (1) definición, (2) arquitectura, (3-6) cuatro sprints de desarrollo, (7) QA y (8) documentación y despliegue.

Distribución de roles

- Esteban Merlano — Líder del proyecto, Backend y Despliegue.
- Keyner Trujillo — Frontend.
- Andres Rangel — Base de datos.
- Jhon Escobar — Aseguramiento de calidad (QA).

Gestión de versiones

Git y GitHub con GitFlow: ramas produccion, develop y una por rol (backend, frontend, base-de-datos, qa). Integración mediante Pull Requests con revisión de código.

Metodología de trabajo

Ágil con sprints semanales, reuniones diarias breves (daily), tablero de tareas en ClickUp y retrospectivas al cierre de cada sprint.

8. Implementación

Descripción de módulos

Autenticación, Experiencias, Reservas, Reseñas, Pagos (Stripe), Solicitudes de guía y Administración.

Integración del sistema

El frontend consume la API REST con Axios; el backend se integra con NeonDB (datos), Stripe (pagos) y OpenFreeMap (mapas). La confirmación de pagos es idempotente (webhook y verificación de sesión).

Evidencias de desarrollo

Repositorio Git con historial por ramas, aplicación desplegada (URLs en la sección 10) y documentación de API en vivo (Swagger).

9. Plan de Pruebas

Pruebas unitarias

Validación de reglas de negocio: control de cupos, rango de puntuación (1-5), restricciones por rol y formato de datos.

Pruebas de integración

Pruebas de extremo a extremo de los endpoints (registro → reserva → pago) ejecutadas contra la base de datos real en Neon.

Pruebas funcionales

22 casos de prueba en total (15 funcionales y 7 de seguridad): login, filtros, reservas con y sin cupos, pagos, reseñas y verificación de guías.

Resultados y validación

100 % de casos exitosos tras correcciones; 3 defectos detectados y cerrados; flujo de pago con Stripe verificado en modo prueba (tarjeta 4242 4242 4242 4242).

10. Despliegue

Infraestructura

Vercel (frontend + backend como funciones serverless) y NeonDB (PostgreSQL). Aplicación en vivo:

- Aplicación: <https://muelle-digital.vercel.app>
- API: <https://muelle-digital-api.vercel.app>
- Documentación API (Swagger): <https://muelle-digital-api.vercel.app/api/docs>

Configuración del servidor

Variables de entorno cifradas en Vercel (DATABASE_URL, JWT_SECRET, claves de Stripe, CLIENT_URL); el backend corre como función serverless (api/index.js + vercel.json) y el CORS se restringe al dominio del frontend.

Manual de instalación

1) Clonar el repositorio. 2) En backend/ y frontend/: npm install. 3) Configurar el archivo .env. 4) Crear la base de datos con npm run db:setup. 5) Ejecutar con npm run dev.

Manual de usuario

Turista: busca, reserva, paga y califica. Guía: publica experiencias y marca el punto de encuentro en el mapa (previa aprobación). Administrador: aprueba guías y gestiona usuarios, experiencias y pagos.

11. Resultados y Evaluación

Resultados obtenidos

Plataforma fullstack funcional y desplegada en producción, con las 13 historias de usuario implementadas, pagos reales en modo prueba y mapas operativos.

Comparación con objetivos

Se cumplieron todos los objetivos específicos: reservas, pagos, reseñas, geolocalización y verificación de guías.

Limitaciones encontradas

Pagos en modo de prueba (no cobro real), sin aplicación móvil nativa y reparto de comisiones por guía (Stripe Connect) planteado como trabajo futuro.

12. Conclusiones y Recomendaciones

Conclusiones

La metodología SDD permitió construir un MVP coherente, seguro y desplegado; el uso de un stack homogéneo (JavaScript en frontend y backend) aceleró el desarrollo y facilitó la integración.

Recomendaciones futuras

Implementar Stripe Connect para comisiones por guía, automatizar pruebas con CI/CD, agregar geolocalización avanzada y desarrollar una aplicación móvil.

13. Referencias Bibliográficas

Documentación oficial consultada (formato APA/IEEE):

- React. (s. f.). React Documentation. <https://react.dev>
- OpenJS Foundation. (s. f.). Node.js & Express Documentation.
- The PostgreSQL Global Development Group. (s. f.). PostgreSQL Documentation.
- Stripe. (s. f.). Stripe API Reference. <https://docs.stripe.com>
- MapLibre / OpenStreetMap. (s. f.). MapLibre GL JS & OSM.
- Vercel & Neon. (s. f.). Documentación de despliegue.

14. Anexos

Código fuente

Repositorio Git con ramas por rol (produccion, develop, backend, frontend, base-de-datos, qa).

Diagramas

Diagrama Entidad-Relación, diagrama de arquitectura y cronograma (incluidos en el documento maestro del proyecto).

Encuestas

No aplica para esta entrega.

Manuales

Manual de usuario y manual de instalación (incluidos en este documento y en los entregables del equipo).

Documentación adicional

Documentación interactiva de la API (Swagger), documento maestro y entregables por rol.